Step 22 - Common Design Mistakes 1
Reference Note 1
Learning Objectives 1
Mistake 1 - Confusing Counter and Timer 1
Incorrect 1
Correct 1
Remember 1
Mistake 2 - Assuming CPU Clock = Timer Clock 1
Incorrect 1
Correct 1
Example 1
Mistake 3 - Assuming Delay Loops Are Timers 1
Incorrect 1
Problems 1
Correct Approach 1
Mistake 4 - Misunderstanding TVAL and CVAL 1
Incorrect 1
Correct 1
Mistake 5 - Using Relative Scheduling for Periodic Events 1
Common Pattern 1
Problem 1
Preferred Pattern 1
Benefit 1
Mistake 6 - Assuming Timer Expiry Means ISR Execution 1
Incorrect 1
Correct 1
Mistake 7 - Misunderstanding IMASK 1
Incorrect 1
Correct 1

Mistake 8 - Misunderstanding ISTATUS 1
Incorrect 1
Correct 1
Mistake 9 - Confusing Pending and Active Interrupts 1
Incorrect 1
Correct 1
Mistake 10 - Assuming Every Core Has Its Own System Counter 1
Incorrect 1
Correct 1
Mistake 11 - Confusing Shared Counters and Per-Core Timers 1
Correct Model 1
Mistake 12 - Thinking CNTVCT Uses Another Clock 1
Incorrect 1
Correct 1
Mistake 13 - Misunderstanding CNTVOFF_EL2 1
Incorrect 1
Correct 1
Mistake 14 - Thinking _EL0 Means EL0 Ownership 1
Incorrect 1
Correct 1
Mistake 15 - Assuming Virtualization Requires New Counters 1
Incorrect 1
Correct 1
Mistake 16 - Forgetting To Enable Interrupt Components 1
Required Chain 1
Mistake 17 - Ignoring Counter Frequency 1
Incorrect 1
Correct 1
Mistake 18 - Worrying Excessively About Rollover 1

Concern 1
Reality 1
Best Practices Summary 1
Architecture Sanity Checklist 1
Key Takeaways 1
Knowledge Check 1
Q1 1
Q2 1
Q3 1
Q4 1
Q5 1
Answer Key 1

Step 22 - Common Design Mistakes

Reference Note

This chapter is based on concepts described in the ARM Generic Timer Architecture documentation.

Over the previous chapters we learned:

- System Counter
- Physical and Virtual Counters
- Physical and Virtual Timers
- TVAL vs CVAL
- Timer Virtualization
- PPIs
- GIC Interaction
- Timer Initialization

This chapter focuses on the mistakes that engineers commonly make when learning, designing, or debugging ARM Generic Timer implementations.

Learning Objectives

After completing this chapter, you should understand:

- Common Generic Timer misconceptions
- Typical debugging mistakes
- Counter vs Timer confusion
- TVAL vs CVAL mistakes
- Physical vs Virtual time confusion
- Interrupt delivery misunderstandings
- Best practices for timer programming

Mistake 1 - Confusing Counter and Timer

One of the most common mistakes.

Incorrect

Counter = Timer

Correct

Component	Purpose
Counter	Measures time
Timer	Generates events using time

Remember

Question	Component
What time is it?	Counter
Has the deadline been reached?	Timer

Mistake 2 - Assuming CPU Clock = Timer Clock

Incorrect

CPU Frequency = System Counter Frequency

Correct

Component	Purpose
CPU Clock	Executes instructions
System Counter	Measures time

These are intentionally independent.

Example

CPU Frequency	System Counter Frequency
800 MHz	24 MHz
2 GHz	24 MHz

Timekeeping remains unchanged.

Mistake 3 - Assuming Delay Loops Are Timers

Incorrect

```
for(i = 0; i < 1000000; i++)
{
}
```

Problems

Dependency	Result

CPU Frequency	Delay changes
Compiler Optimization	Delay may disappear
Processor Type	Delay changes

Correct Approach

Use:

System Counter
Timer Registers

instead of software loops.

Mistake 4 - Misunderstanding TVAL and CVAL

Incorrect

TVAL and CVAL are different timers

Correct

Register	Purpose
TVAL	Relative Time
CVAL	Absolute Time

Both program the same timer.

Mistake 5 - Using Relative Scheduling for Periodic Events

Common Pattern

CurrentCounter + Period

inside every interrupt.

Problem

Interrupt latency accumulates.

This causes:

Scheduler Drift

Preferred Pattern

NextDeadline += Period

using:

CVAL

Benefit

Method	Result
TVAL Relative Scheduling	Drift Accumulates
Absolute Deadlines	Stable Timing

Mistake 6 - Assuming Timer Expiry Means ISR Execution

Incorrect

Timer Expires
↓
ISR Executes

Correct

Step	Action
1	Timer Expires
2	PPI Asserted
3	GIC Marks Pending
4	CPU Accepts Interrupt
5	ISR Executes

Several stages exist between expiry and software execution.

Mistake 7 - Misunderstanding IMASK

Incorrect

IMASK stops timer operation

Correct

Feature	Effect
ENABLE = 0	Timer disabled
IMASK = 1	Interrupt output masked

The timer still:

- Counts
- Compares
- Expires
- Updates ISTATUS

Mistake 8 - Misunderstanding ISTATUS

Incorrect

ISTATUS = 1 means ISR executed

Correct

ISTATUS = 1

means:

Timer Expired

only.

It does **not** guarantee:

- GIC delivery
- CPU acceptance
- ISR execution

Mistake 9 - Confusing Pending and Active Interrupts

Incorrect

Pending = Running

Correct

State	Meaning
Pending	Waiting for CPU
Active	ISR executing

An interrupt can remain pending for many reasons:

- Masked
- Lower priority
- CPU IRQ disabled

Mistake 10 - Assuming Every Core Has Its Own System Counter

Incorrect

Core0 Counter
Core1 Counter
Core2 Counter
Core3 Counter

Correct

Resource	Scope
System Counter	Shared
Physical Timer	Per Core
Virtual Timer	Per Core

Mistake 11 - Confusing Shared Counters and Per-Core Timers

Correct Model

Component	Shared?
System Counter	Yes
Physical Counter View	Shared Time Source
Physical Timer	Per Core
Virtual Timer	Per Core
PPI	Per Core

Mistake 12 - Thinking CNTVCT Uses Another Clock

Incorrect

Virtual Counter requires another clock source

Correct

Virtual Time is derived from:

System Counter
↓
CNTVOFF_EL2
↓
CNTVCT

There is only one underlying time source.

Mistake 13 - Misunderstanding CNTVOFF_EL2

Incorrect

CNTVOFF modifies the System Counter

Correct

Component	Modified?
System Counter	No
CNTPCT	No
CNTVCT	Yes

Only the virtual view changes.

Mistake 14 - Thinking _EL0 Means EL0 Ownership

Incorrect

CNTPCT_EL0

means:

Only EL0 may access it

Correct

The name identifies the architectural register.

Access permissions depend on:

Register
CNTKCTL_EL1

CNTHCTL_EL2
EL Configuration

Mistake 15 - Assuming Virtualization Requires New Counters

Incorrect

One Hardware Counter Per VM

Correct

ARM uses:

One System Counter
+
Offsets

to support many virtual machines efficiently.

Mistake 16 - Forgetting To Enable Interrupt Components

Timer expiry requires more than:

ENABLE = 1

Required Chain

Component	Enabled?
Timer	Yes

PPI	Yes
GIC	Yes
CPU IRQ	Yes

Missing any element breaks interrupt delivery.

Mistake 17 - Ignoring Counter Frequency

Incorrect

Program:

TVAL = 1000

without knowing:

CNTFRQ_EL0

Correct

Time calculations require:

Parameter	Purpose
CNTFRQ_EL0	Ticks per second
Tick Count	Desired interval

Mistake 18 - Worrying Excessively About Rollover

Concern

64-bit Counter Will Overflow

Reality

Example:

Counter Frequency	Approximate Rollover Time
24 MHz	~24,000+ years

For most systems:

Rollover is practically irrelevant.

Best Practices Summary

Recommendation	Reason
Use CVAL for periodic timers	Prevent drift
Use CNTFRQ_EL0 for all conversions	Accurate timing
Know the difference between counter and timer	Correct architecture understanding
Understand Pending vs Active	Easier debugging
Understand Physical vs Virtual Time	Correct virtualization design
Use shared timeline assumptions	Correct SMP behavior

Architecture Sanity Checklist

Before debugging timer problems, verify:

Item
Counter advancing
CNTFRQ known
Timer enabled
Correct CVAL/TVAL
IMASK cleared
PPI enabled
GIC enabled

CPU IRQ enabled

Key Takeaways

Concept	Common Mistake
Counter vs Timer	Treating them as identical
TVAL vs CVAL	Treating them as different timers
IMASK	Assuming it stops timing
ISTATUS	Assuming ISR executed
Virtualization	Assuming separate counters exist
System Counter	Assuming one per core
Interrupt Flow	Assuming expiry immediately runs ISR

Knowledge Check

Q1

What is the difference between a Counter and a Timer?

Q2

Why is CVAL generally preferred for periodic timers?

Q3

Does IMASK stop timer counting?

Q4

Does ISTATUS guarantee ISR execution?

Q5

Does ARM create a hardware counter for each virtual machine?

Answer Key

Question	Answer
Q1	Counter measures time, Timer generates events
Q2	Prevents accumulated drift
Q3	No
Q4	No
Q5	No